

TEMA 2.2

CSS: FUNDAMENTOS Y ESTILOS DE LA WEB

APLICACIONES WEB



Esta obra está bajo una
Licencia CC BY-NC-SA 4.0 Internacional.

Jesús Cáceres Tello
Dpto de Sistemas Informáticos y Computación
Facultad de Informática
Universidad Complutense de Madrid

- **INTRODUCCIÓN**
- SELECTORES Y SINTAXIS
- PROPIEDADES FUNDAMENTALES
- MODELOS DE CAJA (BOX MODEL)
- POSICIONAMIENTO Y FLUJO
- LAYOUTS: FLEXBOX Y GRID
- PRIORIDAD DE ESTILOS
- BUENAS PRÁCTICAS Y RESPONSIVE
- BIBLIOGRAFÍA

OBJETIVOS

- Comprender la función de CSS y su papel frente a HTML en la construcción de páginas web.
- Aplicar reglas de estilo en distintas formas (inline, interno y externo) reconociendo ventajas e inconvenientes.
- Utilizar selectores y pseudoclasas para dar formato a elementos HTML.
- Configurar propiedades fundamentales de color, texto, tipografía, fondos, bordes y tamaños.
- Explicar y aplicar el modelo de caja (Box Model) y su influencia en el diseño web.
- Controlar el flujo y posicionamiento de elementos mediante display, position y, como referencia histórica, float.
- Diseñar maquetaciones modernas utilizando Flexbox y Grid.
- Gestionar la prioridad de estilos entendiendo la cascada, herencia y especificidad en CSS.
- Implementar diseño responsive con media queries adaptado a distintos dispositivos.
- Adoptar buenas prácticas de accesibilidad en CSS, garantizando contraste de colores, legibilidad, estados de interacción visibles y animaciones inclusivas.

CSS: Introducción

- CSS = Cascading Style Sheets (Hojas de Estilo en Cascada).
- Complementa a HTML:
 - HTML = estructura y significado.
 - CSS = diseño y presentación.
- Se aplica a elementos HTML para modificar su aspecto.
- La apariencia visual de una página se define mediante el lenguaje CSS, y suele definirse en un archivo distinto al documento HTML

```
<!-- Sin CSS -->  
<p>Hola mundo</p>
```

```
<!-- Con CSS -->  
<p style="color: blue; font-size: 20px;">Hola mundo</p>
```

ESTRUCTURA vs. ESTILO

Mismo documento HTML mostrado con distintos estilos:

Funciones de orden superior

Decimos que una función es de **orden superior** si acepta otras funciones como parámetro y/o devuelve funciones como resultado. Entre estas funciones podemos citar `map`, `filter` y la familia de funciones de *folding*.

Funciones de orden superior

Decimos que una función es de **orden superior** si acepta otras funciones como parámetro y/o devuelve funciones como resultado. Entre estas funciones podemos citar `map`, `filter` y la familia de funciones de *folding*.

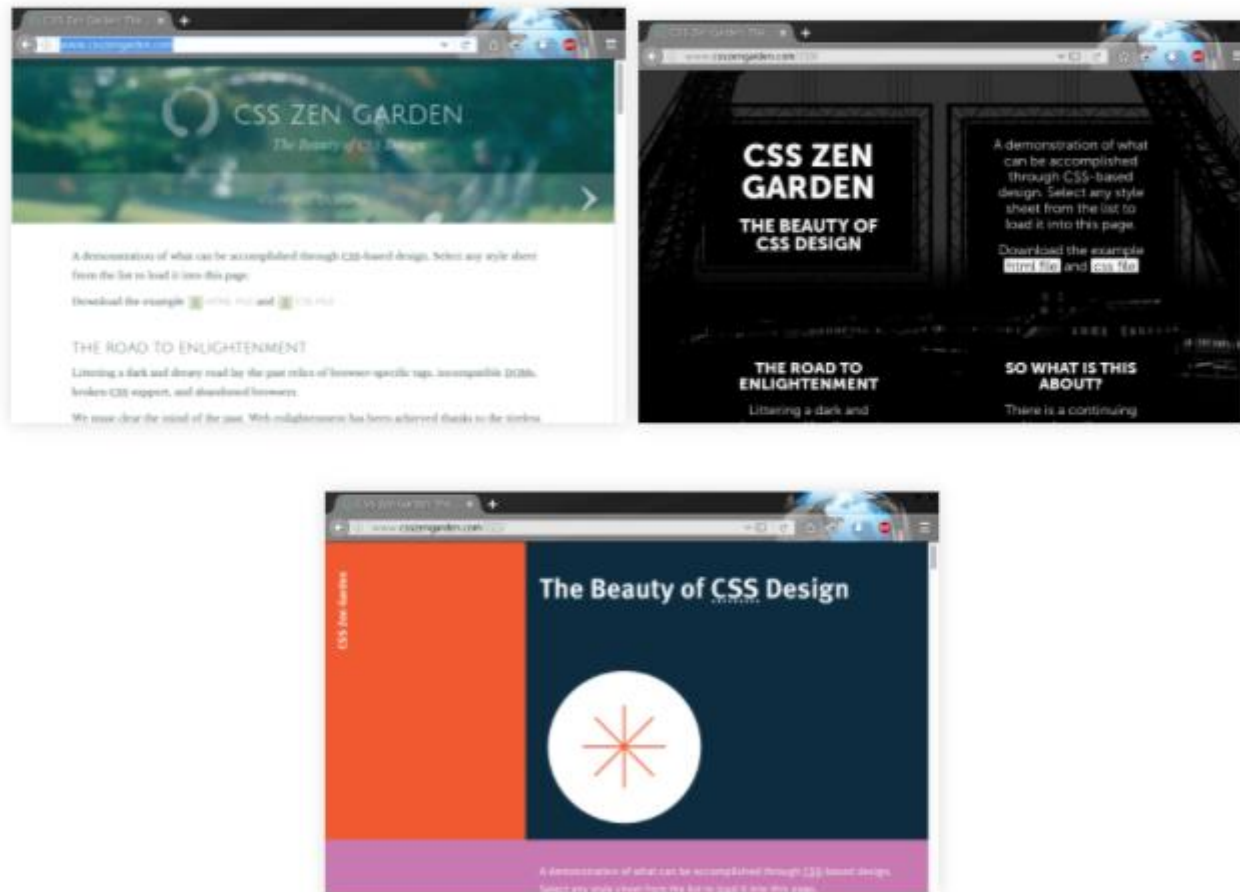
Funciones de orden superior

Decimos que una función es de **orden superior** si acepta otras funciones como parámetro y/o devuelve funciones como resultado. Entre estas funciones podemos citar `map`, `filter` y la familia de funciones de *folding*.

ESTRUCTURA vs. ESTILO (2)

<http://www.csszengarden.com/>

Distintos estilos aplicados a una misma página web.



CSS: Formas de incluir estilos en la web

- Inline → dentro de un atributo style.
- Interno → en <style> dentro de <head>.
- Externo → archivo .css enlazado con <link>

Forma	Ejemplo	Ventajas	Inconvenientes
Inline	<code><p style="color:red;"></code>	Rápido, puntual	No reutilizable, difícil mantenimiento
Interno	<code><style> p {color:red;} </style></code>	Centralizado en un archivo HTML	No sirve para varios HTML
Externo	<code><link rel="stylesheet" href="estilos.css"></code>	Reutilizable, ordenado, profesional	Requiere archivo extra

- Se recomienda **hoja de estilos externa** → permite separar estructura de presentación y mantener un HTML más limpio y semántico.

CSS: Primeros pasos

- Crear un archivo llamado estilos.css.
- Añadir dentro algunas reglas básicas:

```
body {  
  background-color: lightyellow;  
  font-family: Arial, sans-serif;  
}  
  
h1 {  
  color: darkblue;  
}  
  
p {  
  color: darkgreen;  
  font-size: 18px;  
}
```

```
<link rel="stylesheet" href="estilos.css">
```

CONTENIDO

- INTRODUCCIÓN
- **SELECTORES Y SINTAXIS**
- PROPIEDADES FUNDAMENTALES
- MODELOS DE CAJA (BOX MODEL)
- POSICIONAMIENTO Y FLUJO
- LAYOUTS: FLEXBOX Y GRID
- PRIORIDAD DE ESTILOS
- BUENAS PRÁCTICAS Y RESPONSIVE
- BIBLIOGRAFÍA

CSS: Selectores y sintaxis

- `<link rel="stylesheet" href="estilos.css">`
 - Conecta el HTML con la hoja de estilos externa. `rel="stylesheet"` indica la relación, `href` el archivo CSS.
- Selector `body { ... }`
 - Aplica estilos a toda la página.
 - `background-color` → color de fondo (amarillo claro).
 - `font-family` → tipografía general (Arial, si no, sans-serif).
- Selector `h1 { ... }`
 - Cambia solo el título. `color` → lo pinta de azul oscuro.
- Selector `p { ... }`
 - Afecta a los párrafos.
 - `color` → texto en verde oscuro.
 - `font-size` → tamaño de fuente de 18px.

CSS: Estructura básica

- Una regla CSS se compone de:

```
selector {  
  propiedad_1: valor_1;  
  propiedad_2: valor_2;  
  ...  
}
```

- Selector → indica qué elementos HTML serán afectados (ej. p, h1, .clase, #id).
- Propiedad → característica visual que queremos cambiar (ej. color, font-size, background-color).
- Valor → el ajuste que le damos a la propiedad (ej. red, 16px, lightblue).

CSS: Selectores básicos

- **Selector de elemento:** Afecta a todas las etiquetas del mismo tipo

```
p {  
  color: blue;  
}
```

- **Selector de clase:** Afecta a los elementos que tengan el atributo class con el mismo nombre

```
.destacado {  
  background-color: yellow;  
}
```

```
<p class="destacado">Texto importante</p>
```

- **Selector de id:** Afecta al elemento con un id único

```
#titulo{  
  font-size: 24px;  
}
```

```
<h1 id="titulo">Encabezado</h1>
```

CSS: Clases vs. Identificadores

- ID = único en la página. No debe existir dos elementos con el mismo identificador. Se utilizan para estilos únicos
- Clase = reutilizable en varios elementos. Se utilizan para estilos comunes a distintos elementos de la web.

```
<h1 id="principal">Título</h1>  
<p class="nota">Texto destacado</p>
```

CSS: Buenas prácticas con selectores

- Usar clases para estilos reutilizables.
- Un uso correcto de **clases** permite mantener el HTML más limpio y semántico
- Usar id solo cuando el elemento es único en la página.
- No abusar de id en CSS → complica la prioridad de estilos.

Selector	Símbolo	Ejemplo
Elemento	p	p { color: red; }
Clase	.	.destacado { ... }
Id	#	#titulo { ... }

CSS: Selectores universales y combinadores

- Universal

```
{ margin:0; padding:0; }
```

- Descendiente

```
p a { color:red; }
```

- Hijo directo

```
p > a { text-decoration:none; }
```

- Hijo adyacente

```
table + p { font-weight:bold; }  
table ~ p { font-weight:bold; }
```

CSS: Selectores de atributos

- Permiten aplicar estilos a elementos según el valor de sus atributos.
- Usado en formularios, enlaces, iconos, menús dinámicos.

```
img[src="icono.png"] { ... }      /* valor exacto */  
a[href^="https"] { ... }         /* empieza con https */  
a[href$=".php"] { ... }          /* termina en .php */  
a[href*="server"] { ... }       /* contiene "server" */
```

- Idea clave: permiten ser muy específicos sin necesidad de clases adicionales.
- Accesibilidad: evita basar estilos solo en atributos técnicos poco descriptivos (ej. extensiones .php, .asp), ya que puede confundir el propósito del enlace.

```
<a href="https://ejemplo.com">Enlace seguro</a>  
<a href="archivo.php">Script PHP</a>
```

CSS: Pseudoclasas de interacción

- Definen un estado especial de un elemento, normalmente dinámico.
- Se aplica normalmente para resaltar enlaces, botones o inputs en interacción.

```
a:link      { color: blue; }      /* enlace no visitado */
a:visited   { color: purple; }    /* enlace visitado */
a:hover     { color: red; }       /* al pasar el ratón */
a:active    { color: green; }     /* al hacer clic */
input:focus { border: 2px solid orange; }
```

- Mejoran la interacción sin necesidad de JavaScript.
- Accesibilidad: nunca elimines **:focus** en formularios o en enlaces, los usuarios con teclado o lectores de pantalla lo necesitan para navegar.

CSS: Pseudoclasas estructurales

- Aplican estilos en función de la posición del elemento dentro del DOM.
- Muy usado en tablas y listas

```
li:first-child { font-weight: bold; }  
li:last-child  { color: red; }  
li:nth-child(2) { background: lightgray; }  
table tr:nth-child(even) { background:#eee; } /* filas pares */  
table tr:nth-child(odd)  { background:#ddd; } /* filas impares */
```

- Usar estilos alternos mejora la legibilidad (filas pares/impares en tablas).

```
<ul>  
  <li>Elemento 1</li>  
  <li>Elemento 2</li>  
  <li>Elemento 3</li>  
</ul>
```

CSS: Pseudoelementos

- Permiten aplicar estilos a **partes específicas del contenido** o generar contenido adicional.
- Se utilizan generalmente para decoración, iconos, primeras letras estilizadas.
- Principales pseudoelementos:
 - **::first-letter** → estilizar la primera letra.
 - **::first-line** → estilizar la primera línea de un párrafo.
 - **::before** y **::after** → insertar contenido decorativo.
 - **::selection** → cambiar el estilo del texto seleccionado por el usuario.

CSS: Pseudoelementos

```
p::first-letter {
  font-size: 2em;
  font-weight: bold;
  color: darkred;
}

p::first-line {
  text-transform: uppercase;
}

h1::before {
  content: "★ ";
  color: gold;
}

h1::after {
  content: " ◆";
  color: lightgray;
}

/* Personalizar el texto seleccionado
por el usuario */
::selection {
  background-color: navy;
  color: white;
}
```

- Usar pseudoelementos sólo para decoración (flechas, adornos, iconos).
- Nunca poner contenido esencial. Un lector de pantalla puede NO leerlo
- Para iconos con ::before y ::after, acompañarlos con texto alternativo real en HTML si transmiten información.
- Asegurar contrastes suficientes entre el color de fondo y el texto seleccionado en ::selection
- No usar colores que desaparezcan con algunos tipos de daltonismo (p.e., rojo sobre verde)
- Probad selección tanto con ratón como con teclado.

★ Asignatura AW. Pseudoelementos ★

LOREM IPSUM DOLOR SIT AMET, CONSECTETUER ADIPISCING ELIT. MAECIPURUS **lectus malesuada libero, sit amet commodo magna eros quis urna.** Nunc viverra imperdiet enim dolor sit a nibh in interdum aenean ornare vestibulum. Donec ullamcorper nulla non metus auctor fringilla. Vestibulum id ligula nec dolor. Maecenas tempus iaculis nunc. Mauris egestas. Proin pharetra nonummy pede. Mauris et orci. Aenean nec lorem. In porttitor. Donec a metus a elit. Ut nonummy. Pretium mattis, nunc. Mauris eget neque at sem venenatis eleifend. Ut nonummy.

CSS: Wave. Extensión de Chrome

LOREM IPSUM DOLOR SIT AMET, CONSECTETUR ADIPISCING ELIT. VIVAMUS **AC MAGNA EGET URNA CURSUS PLACERAT.**
euismod, nunc vel tincidunt tempus, augue risus.

The image displays four screenshots of the Wave web accessibility evaluation tool interface, which is powered by WebAIM. The tool is used to evaluate the accessibility of web pages.

- Summary:** Shows overall results. 0 Errors, 1 Alerts, 1 Structural Elements, 0 ARIA. A message states: "Congratulations! No errors were detected! Manual testing is still necessary to ensure compliance and optimal accessibility."
- Details:** Shows specific issues. 1 Alerts (1 X No page regions), 1 Features (1 X Language), 1 Structural Elements (1 X Heading level 1).
- Structure:** Shows the document structure. The first heading is

★ Asignatura AW.

 followed by "Pseudoelementos".
- Contrast:** Shows contrast settings. Foreground Hex Value: #0000FF, Background Hex Value: #FFFFFF. Contrast Ratio: 8.59:1. Text Size: Normal. WCAG AA: Pass, WCAG AAA: Pass. A note states: "No contrast errors were detected in the page. Manual testing is necessary to test for other potential contrast issues."

**Pasa el contraste AA y AAA con 8.59:1
... Algo va mal ...**

CSS: Contrast Checker

 webaim.org/resources/contrastchecker/

- Las herramientas automáticas suelen tener limitaciones en las comprobaciones de accesibilidad.
- Siempre hay que combinar análisis automático con pruebas manuales

Contrast Checker

[Home](#) > [Resources](#) > Contrast Checker

Foreground
Hex Value
#FF0000
Color Picker
Alpha
1
Lightness

Background
Hex Value
#00FF00
Color Picker
Lightness

Contrast Ratio
2.91:1
[permalink](#)

Normal Text

WCAG AA: **Fail**

WCAG AAA: **Fail**

The five boxing wizards jump quickly.

Large Text

WCAG AA: **Fail**

WCAG AAA: **Fail**

The five boxing wizards jump quickly.

Graphical Objects and User Interface Components

WCAG AA: **Fail**

★
Text Input

CSS: Accesibilidad por teclado

- Muchas personas navegan solo con teclado (tabulador, espacio, enter, flechas).
- También afecta a usuarios de lectores de pantalla y personas con movilidad reducida.
- En CSS podemos facilitar (o bloquear) esta accesibilidad.
- Mantener siempre visible el foco
- Usar `:focus-visible` para mostrar foco solo cuando se usa teclado (evita “parpadeos” al hacer clic con ratón).
- Respetar el orden natural del DOM → el foco seguirá esa secuencia.
- No desactivar `outline: none`; sin alternativa clara y accesible.
- Asegurar que los elementos interactivos (links, botones, inputs) tengan área de interacción suficiente (mín. 44x44px).

```
a:focus, button:focus {  
  outline: 2px solid orange; /* o un estilo personalizado */  
}
```

```
a:focus, button:focus {  
  outline: none; /* o un estilo personalizado */  
}
```

CSS: Accesibilidad por teclado

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <title>Accesibilidad por teclado</title>

</head>
<body>
  <h1>Demo de foco con teclado</h1>
  <p>
    Usa <strong>TAB</strong> para moverte entre los
    elementos interactivos:
  </p>
  <a href="#">Enlace 1</a>
  <a href="#">Enlace 2</a>
  <button>Botón</button>
  <input type="text" placeholder="Escribe aquí...">
  <a href="#">Enlace final</a>
</p>
  Observa cómo cambia el estilo al recibir
<code>:focus</code>.<br>
  ✗ Si quitas <code>outline</code>, perderías la
  referencia visual.
</p>
</body>
</html>
```

```
style>
  body {
    < font-family: Arial, sans-serif;
    padding: 2rem;
  }
  a, button, input {
    margin: 10px;
    padding: 10px 15px;
    border: 1px solid #ccc;
    text-decoration: none;
    display: inline-block;
  }
  /* ESTILO DE FOCUS ACCESIBLE */
  a:focus, button:focus, input:focus {
    outline: 3px solid orange;
    outline-offset: 3px;
    background-color: #fff7e6;
  }
  /* Para comparar: estilo hover */
  a:hover, button:hover {
    background-color: lightblue;
  }
</style>
```

Demo de foco con teclado

Usa **TAB** para moverte entre los elementos interactivos:

Enlace 1

Enlace 2

Botón

Escribe aquí...

Enlace final

- Observa cómo cambia el estilo al recibir `:focus`.
✗ Si quitas `outline`, perderías la referencia visual.

CSS: Ejercicio

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <title>Noticias del día</title>
  <link rel="stylesheet" href="estilos.css">
</head>
<body>
  <h1 id="principal">Noticias de última hora</h1>

  <div class="noticia destacada">
    <h2 class="titulo">Nueva misión espacial</h2>
    <p>La agencia espacial ha lanzado un nuevo satélite...</p>
    <a href="https://example.com/mision" class="leer-mas">Leer más</a>
  </div>

  <div class="noticia">
    <h2 class="titulo">Tecnología y educación</h2>
    <p>Las universidades apuestan por la inteligencia artificial...</p>
    <a href="https://example.com/educacion" class="leer-mas">Leer más</a>
  </div>

  <div class="noticia">
    <h2 class="titulo">Deportes: final de liga</h2>
    <p>El equipo local se corona campeón tras una temporada intensa...</p>
    <a href="resumen.php" class="leer-mas">Leer más</a>
  </div>
</body>
</html>
```

CSS: Ejercicio

- Dado el código de estructura anterior, diseña un bloque de noticias en el que:
 - El primer párrafo de cada noticia aparezca en negrita.
 - Los enlaces cambien de color cuando el usuario pase el ratón por encima y también cuando se navegue con el teclado.
 - Añade un icono decorativo antes de cada título de noticia.
 - Haz que el primer carácter de cada párrafo sea más grande y destaque visualmente.
 - Personaliza la apariencia del texto cuando es seleccionado por el usuario.
 - Alterna el color de fondo de las noticias para diferenciarlas visualmente.

ENTREGA EN EL CAMPUS
Ejercicio de clase (estilos.css)

CSS: Ejercicio. Accesibilidad

- ¿Qué sucede si el icono decorativo aporta información importante y no solo estética?
- ¿Cómo ayudas a los usuarios que navegan con teclado a saber en qué enlace están?
- ¿Qué problema existe si el único criterio para diferenciar noticias o enlaces es el color?

ENTREGA EN EL CAMPUS

Ejercicio de clase (como comentarios en el archivo css)

CONTENIDO

- INTRODUCCIÓN
- SELECTORES Y SINTAXIS
- **PROPIEDADES FUNDAMENTALES**
- MODELO DE CAJA (BOX MODEL)
- POSICIONAMIENTO Y FLUJO
- LAYOUTS: FLEXBOX Y GRID
- PRIORIDAD DE ESTILOS
- BUENAS PRÁCTICAS Y RESPONSIVE
- BIBLIOGRAFÍA

CSS: Colores en la web

- Formas de especificar colores en CSS
 - Por nombre: red, blue, darkgreen (limitados, poco flexibles).
 - Hexadecimal: #FF0000, #00FF00, #0000FF.
 - RGB: rgb(255, 0, 0) (valores 0–255).
 - RGBA: rgba(255, 0, 0, 0.5) → incluye transparencia.
 - HSL: hsl(120, 100%, 50%) → más intuitivo: tono, saturación y luminosidad.
 - HSLA: igual que HSL pero con canal alfa (transparencia).

```
h1 {  
  color: darkblue;    /* nombre */  
}  
p {  
  color: #006400;     /* hexadecimal */  
}  
.highlight {  
  color: rgb(200, 50, 50); /* RGB */  
  background-color: rgba(255, 200, 200, 0.6); /* RGBA con transparencia */  
}  
.note {  
  color: hsl(200, 70%, 40%); /* HSL */  
}
```

- Títulos en azul oscuro.
- Párrafos en verde oscuro.
- Textos destacados en rojo con fondo rosado semitransparente.
- Notas en un azul saturado.

CSS: Accesibilidad de los colores. Consejos

- No usar solo el color para transmitir información (ej. “los errores salen en rojo”)
 - Añadir iconos, texto o estilos adicionales.
- Mantener siempre un contraste mínimo entre texto y fondo:
 - Texto normal: 4.5:1 (AA)
 - Texto grande (≥ 18 pt o 14pt bold): 3:1 (AA)
- Verificar contraste con herramientas como WebAIM Contrast Checker o WAVE.
- Recordar el daltonismo: rojo-verde y azul-amarillo son combinaciones problemáticas.
- Preferir valores relativos en fuentes y colores accesibles para mejorar escalabilidad.

CSS: Color Pickers



https://www.w3schools.com/colors/colors_picker.asp

HTML Color Picker

< Previous Next >

Pick a Color:



Or Enter a Color:

Color value OK

Or Use HTML5:

Selected Color:

Black Text

Shadow

White Text

Shadow

#0099ff

rgb(0, 153, 255)

hsl(204, 100%, 50%)

Lighter / Darker:

100%	#ffffff
95%	#e6f5ff
90%	#ccebff
85%	#b3e0ff
80%	#99d6ff
75%	#80ccff
70%	#66c2ff
65%	#4db8ff
60%	#33adff
55%	#1aa3ff
50%	#0099ff
45%	#008ae6
40%	#007acc

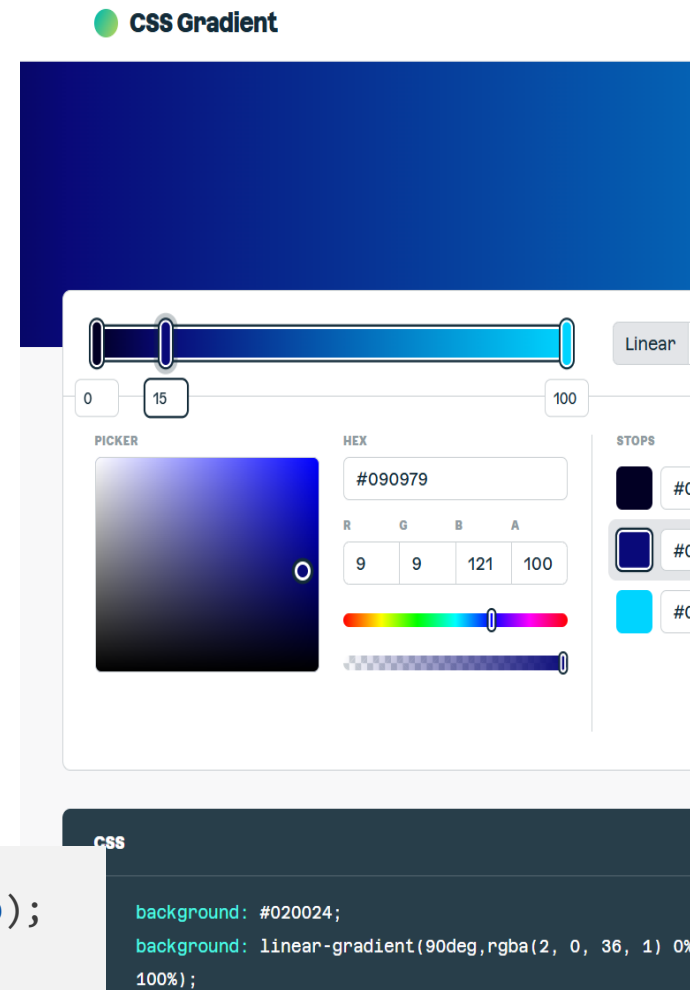
COLORZILLA

ADVANCED COLORFUL GOODIES

CSS: Gradientes

- Los gradientes son soportados por versiones recientes de los navegadores (Chrome ≥ 26 , Firefox ≥ 16 , MS Explorer ≥ 10)
- Herramienta: [CSS Gradient](#)
- Evitar gradientes con poco contraste si hay texto encima.
- Comprobar la legibilidad del texto sobre fondos con gradiente.
- Para paletas, usar herramientas que permitan verificar contraste automáticamente (ej. Coolers con check de WCAG integrado).

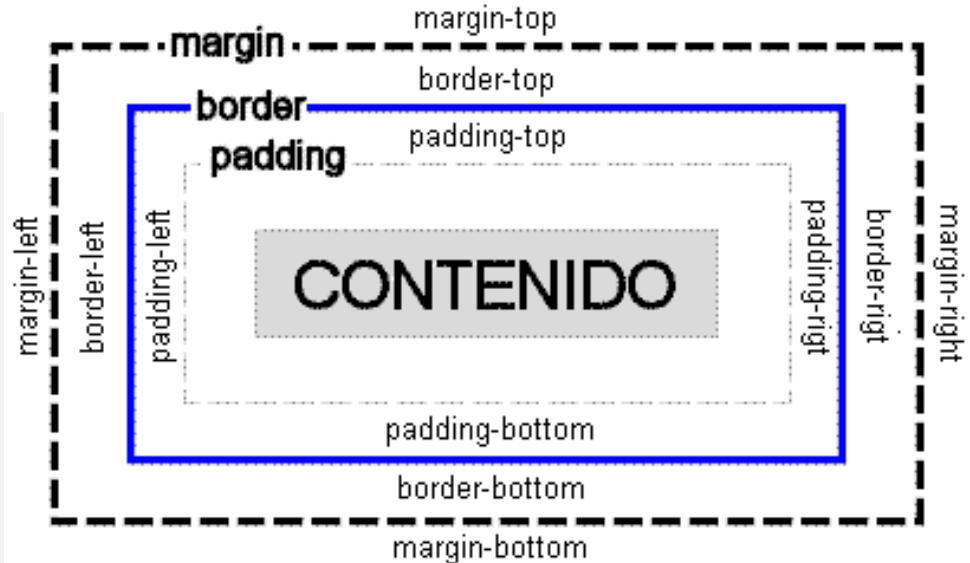
```
.box1 {  
  background: linear-gradient(to right, #ff7e5f, #feb47b);  
}  
  
.box2 {  
  background: radial-gradient(circle, #4facfe, #00f2fe);  
}
```



CSS: Bordes

- Propiedades principales:
 - **border-width**: grosor del borde (thin, medium, thick, px, em).
 - **border-style**: estilo (solid, dashed, dotted, double, groove, ridge, none).
 - **border-color**: color del borde.
 - **border-radius**: esquinas redondeadas.
- Se pueden definir individualmente o con la propiedad abreviada:

```
.box1 {  
  border: 2px solid black;  
}  
.box2 {  
  border: 3px dashed darkred;  
}  
.box3 {  
  border: 4px dotted green;  
  border-radius: 10px;  
}  
.box4 {  
  border: 5px double navy;  
  border-radius: 50%; /* círculo si la caja es cuadrada */  
}
```

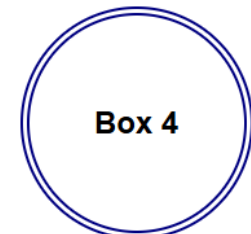
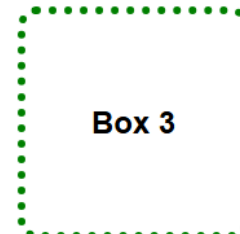
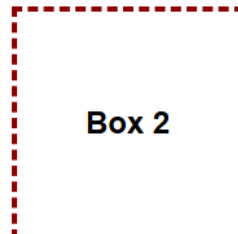
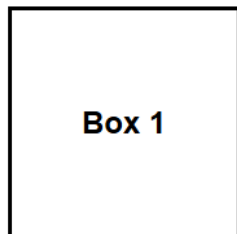


CSS: Bordes

```
.container {  
  display: flex;  
  gap: 20px;  
  flex-wrap: wrap;  
  /* por defecto flex-direction = row */  
}  
  
.caja {  
  width: 120px;  
  height: 120px;  
  display: flex;  
  align-items: center;  
  justify-content: center;  
  background: white;  
  font-weight: bold;  
}
```

```
<h1>Ejemplo de Bordes en CSS</h1>  
<div class="container">  
  <div class="caja box1">Box 1</div>  
  <div class="caja box2">Box 2</div>  
  <div class="caja box3">Box 3</div>  
  <div class="caja box4">Box 4</div>  
</div>
```

★ Ejemplo de Bordes en CSS ✨



CSS: Bordos accesibles

- Mantener visible el foco de teclado

```
input:focus, a:focus {  
  border: 2px solid orange; /* foco accesible */  
  outline: none;           /* solo si das alternativa */  
}
```

- Usar grosor y contraste suficiente

```
button {  
  border: 2px solid #000; /* mejor que 1px gris claro */  
}
```

- No depender sólo del color del borde

```
<input class="error" aria-describedby="msg-error">  
<span id="msg-error">Por favor, rellena este campo</span>
```

```
.error { border: 2px solid red; }
```

- Utilizar border-radius mejora la usabilidad táctil

```
.btn {  
  border: 2px solid #333;  
  border-radius: 20px; /* estilo pill button */  
  padding: 12px 20px; /* área clicable mayor */  
}
```

CSS: Márgenes y Padding

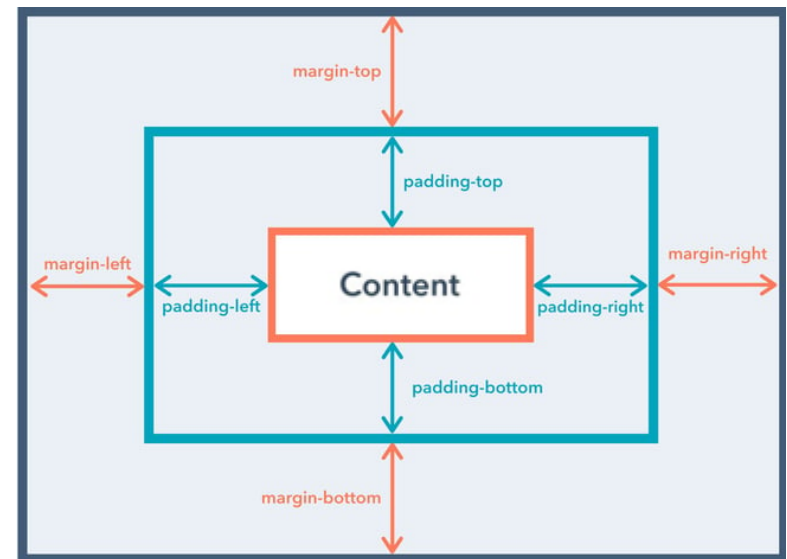
- **margin:** espacio externo, separa la caja de otras cajas.
- **padding:** espacio interno, separa el contenido del borde de la caja.

```
.caja1 {  
  background: lightblue;  
  margin: 20px;    /* separación externa */  
  padding: 10px;   /* espacio interno */  
}  
  
.caja2 {  
  background: lightgreen;  
  margin: 40px 10px; /* arriba/abajo - izquierda/derecha */  
  padding: 5px 30px; /* arriba/abajo - izquierda/derecha */  
}
```

- Shorthands (igual en padding):
 - **margin: 20px;** -> mismo valor en los 4 lados.
 - **margin: 10px 20px;** → vertical | horizontal.
 - **margin: 10px 20px 30px 40px;** → arriba | derecha | abajo | izquierda.

CSS: Márgenes accesibles

- Usar **padding** suficiente mejora la legibilidad (texto no pegado al borde).
- **margin** ayudan a separar bloques de contenido y evitan confusión visual.
- Evitar márgenes excesivos, obligan a hacer scroll innecesario, sobre todo en móviles.
- En elementos interactivos (botones, links), usar padding para agrandar la zona clicable (mínimo recomendado: 44x44px según WCAG 2.1).



CSS: EJEMPLO

```
<p> Esto es un párrafo normal</p>
<p class = "importante">
    Esto es muy importante!!!
</p>
<p>Volvemos al estilo de párrafos anterior</p>
```

```
p.importante {
    background-color: rgb(255, 200, 200);
    border: 1px dashed rgb(200, 0, 0);
    border-left: 5px solid rgb(200, 0, 0);
    margin-left: 20px;
    padding: 10px 30px;
    font-weight: bold;
    width: 200px;
}
```

Esto es un párrafo normal

Esto es muy importante!!!

Volvemos al estilo de párrafos anterior

CSS: Unidades de medida

- **pt**: Medida fija tradicional para impresión, no escalable en web.
- **%**: Relativa al tamaño del contenedor, útil para adaptabilidad pero puede ser difícil de manejar.
- **em**: Relativo al tamaño de fuente del contenedor padre, pero puede acumularse si hay varios niveles de anidamiento.
- **px**: Medida fija en píxeles, precisa pero no adaptable a pantallas de diferentes tamaños.
- **rem**: Relativa al tamaño de fuente raíz, escalable y más fácil de manejar que em en términos de consistencia.

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <meta name="viewport"
    content="width=device-width, initial-scale=1.0">
  <title>Ejemplo de Unidades de Medida</title>
  <style>
    body { font-size: 16px; /* Establece la base para rem */ }
    .texto-px { font-size: 20px; }
    .texto-pt { font-size: 14pt; }
    .texto-em { font-size: 1.5em; }
    .texto-rem { font-size: 1.5rem; }
    .texto-percent { font-size: 120%; }
  </style>
</head>
<body>
  <p class="texto-px">Texto con 20px.</p>
  <p class="texto-pt">Texto con 14pt.</p>
  <p class="texto-em">Texto con 1.5em.</p>
  <p class="texto-rem">Texto con 1.5rem.</p>
  <p class="texto-percent">
    Texto con 120% del tamaño del padre.
  </p>
</body>
</html>
```

CSS: Atributos básicos de texto

- **color**: color del texto
- **font-size**: tamaño de la letra (px, em, rem, %)
- **font-weight**: grosor (normal, bold, 100–900)
- **font-style**: estilo (normal, italic, oblique)
- **text-align**: alineación (left, center, right, justify)
- **text-transform**: mayúsculas/minúsculas (uppercase, lowercase, capitalize)
- **text-decoration**: decoraciones (none, underline, line-through, overline)

CSS: Textos accesibles

- Mantener tamaño de fuente legible (mínimo 16px recomendado).
- Usar unidades relativas (em, rem) para que el texto se adapte al zoom.
- Evitar justificar en exceso: puede crear ríos de espacio en dislexia.
- No quitar subrayado de enlaces sin alternativa (ej. color + :hover claro).
- Usar line-height adecuado (1.4–1.6) para mejorar lectura.

CSS: Tipografías nuevas

- Fuentes del sistema (seguras y preinstaladas)

```
body {  
  font-family: Arial, Helvetica, sans-serif;  
}
```

- Google Fonts (importadas desde la web)

- En el <head> de tu HTML:

```
<link href="https://fonts.googleapis.com/css2?family=Roboto&display=swap" rel="stylesheet">
```

- Y en CSS: body { font-family: 'Roboto', sans-serif;}

```
body {  
  font-family: 'Roboto', sans-serif;  
}
```

- @font-face (fuentes locales o descargadas)

```
@font-face {  
  font-family: 'MiFuente';  
  src: url('fonts/MiFuente.woff2') format('woff2'),  
       url('fonts/MiFuente.woff') format('woff');  
}  
h1 {  
  font-family: 'MiFuente', sans-serif;  
}
```

CSS: Tipologías accesibles

- Usar fuentes claras y legibles (sans-serif en pantalla, serif solo en títulos o impresiones,...).
- Definir siempre fuentes de reserva:

```
font-family: 'Roboto', Arial, sans-serif;
```
- Mantener un tamaño base $\geq 16\text{px}$ y permitir escalado (usar em o rem).
- Evitar fuentes decorativas en bloques largos de texto: solo para títulos breves.
- Revisar compatibilidad y licencia de fuentes externas (ej. Google Fonts, @font-face).
- Evitar usar solo mayúsculas en textos largos: dificulta la lectura.
- Procurar buen contraste entre texto y fondo, incluso si la tipografía es clara.

CSS: Otros atributos de texto

- **letter-spacing**: espacio entre letras
- **word-spacing**: espacio entre palabras
- **text-indent**: sangría en la primera línea
- **text-shadow**: sombra del texto
- **white-space**: controla saltos de línea (normal, nowrap, pre, pre-wrap)
- **overflow-wrap**: controla cortes de palabra (normal, break-word, anywhere)

```
<style>
  h1 {
    letter-spacing: 2px;           /* más separación entre letras */
    text-shadow: 2px 2px 5px gray; /* sombra ligera */
  }
  p {
    text-indent: 20px;            /* sangría en primera línea */
    word-spacing: 5px;           /* espacio entre palabras */
    overflow-wrap: break-word;    /* evita que las palabras largas rompan el layout */
  }
</style>
```

Ejemplos otros atributos de texto

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Maecenas porttitor congue massa. Fusce posuere, magna sed pulvinar ultricies, purus lectus malesuada libero, sit amet commodo magna eros quis urna.

CSS: Dimensiones y Unidades

- Útiles para un diseño más flexible sin romper el layout:
 - **width / height**: ancho y alto del elemento.
 - **min-width / max-width**: ancho mínimo y máximo
 - **min-height / max-height**: alto mínimo y máximo.
- Las unidades de medida pueden ser las siguientes:
 - **Absolutas**
 - px (píxeles), cm, mm, in: poco flexibles, dependen de la pantalla.
 - **Relativas a fuente**
 - em: relativo al font-size del elemento padre.
 - rem: relativo al font-size raíz (html).
 - **Relativas al contenedor o viewport**
 - % → relativo al tamaño del contenedor.
 - vw → porcentaje del ancho de la ventana.
 - vh → porcentaje del alto de la ventana.

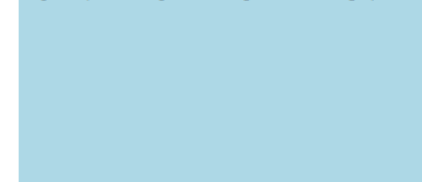
CSS: Dimensiones y Unidades. Consejos

- Usar unidades relativas (em, rem, %, vh, vw): permiten zoom sin romper el diseño.
- Evitar layouts rígidos en px: dificultan la adaptación a pantallas pequeñas.
- min-width y max-width evitan textos demasiado anchos (mejor lectura) o demasiado comprimidos.
- Mantener tamaños de botones y campos de formulario cómodos (mín. 44px de alto).

Ejemplo con .caja1, .caja2 y .texto

```
.caja1 {  
  width: 300px;      /* fijo */  
  height: 150px;  
}  
.caja2 {  
  width: 80%;        /* relativo al contenedor */  
  min-width: 200px; /* nunca más pequeño que 200px */  
  max-width: 600px; /* nunca más grande que 600px */  
}  
.texto {  
  font-size: 1rem;   /* relativo a raíz */  
  line-height: 1.5;  
}
```

Caja 1 (ancho fijo de 300px, alto 150px)



Caja 2 (ancho relativo al contenedor, entre 200px y 600px)



Este es un párrafo con la clase .texto. El tamaño de fuente es relativo a la raíz (1rem) y el interlineado es 1.5, lo que mejora la legibilidad. Prueba a cambiar el ancho de la ventana para ver cómo se adapta la .caja2.

CSS: Fondos o Backgrounds

- **background-color**: color de fondo.
- **background-image**: imagen de fondo.
- **background-repeat**: repetición (repeat, no-repeat, repeat-x, repeat-y).
- **background-position**: posición (left, center, right, valores en % o px).
- **background-size**: tamaño (auto, cover, contain, valores en % o px).
- **background-attachment**: desplazamiento (scroll, fixed).
- **background**: propiedad abreviada.

CSS: Fondos o Backgrounds

```
<style>
  body {
    background-color: #f5f5f5;
  }
  .banner {
    height: 200px;
    background: url("imagenes/logo.jpg") center/cover no-repeat;
    display: flex;
    align-items: center;
    justify-content: center;
    color: white;
  }
  .card {
    background: linear-gradient(to right, #ff7e5f, #feb47b);
    padding: 20px;
    color: white;
  }
</style>
```

```
<body>
  <body>
    <h1>Fondos en CSS</h1>
    <section class="banner">
      <h2 class="overlay">Bienvenido</h2>
    </section>
    <section class="card">
      <h2>Tarjeta destacada</h2>
      <p>
        Este es un ejemplo de tarjeta con un gradiente como fondo.
        El texto debe ser legible en cualquier situación.
      </p>
    </section>
  </body>
```

CSS: Herencia de propiedades

- Algunas propiedades se heredan automáticamente de los elementos padre a los elementos hijos. (Ejemplo: tipografía y color de texto)
- Propiedades que se heredan
 - Color
 - font-family, font-size, font-style, font-weight
 - line-height, letter-spacing, word-spacing
 - visibility
- Propiedades que NO se heredan:
 - margin, padding, border, backgroundwidth, height, position, display

CSS: Herencia de propiedades

- Existen formas de forzar esa herencia o resetear el estilo:

```
p {  
  color: inherit; /* fuerza heredar el color del padre */  
  margin: initial; /* vuelve al valor por defecto del navegador */  
  border: unset; /* elimina borde y sigue reglas de herencia */  
}
```

- La herencia de font-size y color facilita mantener coherencia visual, es decir, buena legibilidad.
- No depender solo de herencia para estilos críticos; comprobar contraste en cada nivel.
- Usar herencia con cuidado en formularios: los inputs no heredan siempre el font-family; declararlo explícitamente si se necesita.

CSS: Ejercicio

Diseña una **página sencilla de noticias** que incluya al menos **dos tarjetas** con contenido (título, texto, imagen y botón).

Aplicar todo lo aprendido en el bloque de Propiedades Fundamentales:

1. Colores y contraste

1. Elige un esquema de colores accesible (texto legible sobre fondo).
2. Usa un gradiente o imagen de fondo en el encabezado de la página.

2. Bordes y bordes redondeados

1. Cada tarjeta debe tener un borde diferenciado (solid, dashed, dotted o double).
2. Redondea las esquinas de los botones (border-radius).

3. Márgenes y padding

1. Separa claramente las tarjetas entre sí (margin).
2. Añade padding interno al texto dentro de las tarjetas para mejorar la legibilidad.

4. Texto y tipografía

1. Título en mayúsculas (text-transform: uppercase) con una fuente de Google Fonts.
2. Texto del párrafo con interlineado adecuado (line-height) y tipografía legible.
3. Evita subrayado en enlaces, pero asegúrate de que cambien con :hover y :focus.

5. Dimensiones y unidades

1. Fija un ancho máximo (max-width) para las tarjetas.
2. Usa rem o % en lugar de px siempre que sea posible.

6. Fondos

1. Al menos una tarjeta debe tener fondo con gradiente.
2. Otra tarjeta puede tener una imagen de fondo con overlay para el texto.

7. Herencia de propiedades

1. Define color y font-family en el body y observa cómo los hijos los heredan.
2. En al menos un elemento, fuerza el uso de inherit o unset para mostrar su efecto.

CSS: Ejercicio

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <title>Noticias</title>
  <link rel="stylesheet" href="estilos.css">
</head>
<body>
  <header>
    <h1>Noticias de hoy</h1>
  </header>
  <main>
    <article class="tarjeta">
      <h2>Título de la noticia 1</h2>
      <p>
        Este es el resumen de la noticia número 1. Aquí se explica brevemente
        de qué trata la información principal.
      </p>
      <a href="#" class="btn">Leer más</a>
    </article>
    <article class="tarjeta">
      <h2>Título de la noticia 2</h2>
      <p>
        Este es el resumen de la noticia número 2. Aquí se explica brevemente
        de qué trata la información principal.
      </p>
      <a href="#" class="btn">Leer más</a>
    </article>
  </main>
</body>
</html>
```

CONTENIDO

- INTRODUCCIÓN
- SELECTORES Y SINTAXIS
- PROPIEDADES FUNDAMENTALES
- **MODELO DE CAJA (BOX MODEL)**
- POSICIONAMIENTO Y FLUJO
- LAYOUTS: FLEXBOX Y GRID
- PRIORIDAD DE ESTILOS
- BUENAS PRÁCTICAS Y RESPONSIVE
- BIBLIOGRAFÍA

CSS: Modelo de Caja (Box Model)

- En CSS, **cada elemento HTML se representa como una caja rectangular**, formada por 4 áreas:
- **Content**: el contenido real (texto, imágenes).
- **Padding**: espacio interno entre contenido y borde.
- **Border**: línea que envuelve el contenido + padding.
- **Margin**: espacio externo que separa un elemento de otro.

```
.caja {  
  width: 200px;  
  padding: 20px;  
  border: 5px solid darkblue;  
  margin: 15px;  
  background-color: lightyellow;  
}
```



CSS: Modelo de Caja (Box Model)

- width + padding izquierdo + padding derecho + borde izquierdo + borde derecho
- Aunque width es 200px, la caja ocupa en realidad: $200 + 20 + 20 + 5 + 5 = 250\text{px}$ de ancho total + margin de 15px por ambos lados = 280px

CSS: Modelo de Caja (Box Sizing)

- Para simplificar los cálculos se puede usar la propiedad **box-sizing**
- Valores:
 - **content-box** (por defecto):
 - width y height: sólo aplican al contenido
 - padding y border se suman al tamaño
 - **border-box** (recomendado):
 - width y height: incluyen contenido + padding + border
 - Los tamaños son más predecibles (layouts responsive)
 - **padding-box** (obsoleto):
 - No tiene soporte. Mejor es no utilizar.

```
/* Recomendación práctica */  
*, *::before, *::after {  
  box-sizing: border-box;  
}
```

CSS: Modelo de Caja accesible

- Consistencia en el tamaño de las cajas
 - Usar box-sizing: border-box; : facilita layouts responsive y evita que elementos se solapen o queden ocultos.
- Control del ancho de los textos
 - Usar max-width en cajas de texto: mejora la legibilidad (líneas no demasiado largas).
- Evitar cajas rígidas en píxeles
 - Es mejor utilizar unidades relativas (em, rem, %, vw, vh): permiten zoom y adaptabilidad.
- Espaciado predecible
 - Márgenes y padding equilibrados ayudan a mantener jerarquía visual que es fundamental en usuarios con baja visión o dificultades cognitivas.

```
*, *::before, *::after {  
  box-sizing: border-box; /* más accesible y predecible */  
}  
.article {  
  max-width: 65ch; /* caja de texto cómoda para leer */  
}
```

CONTENIDO

- INTRODUCCIÓN
- SELECTORES Y SINTAXIS
- PROPIEDADES FUNDAMENTALES
- MODELO DE CAJA (BOX MODEL)
- **POSICIONAMIENTO Y FLUJO**
- LAYOUTS: FLEXBOX Y GRID
- PRIORIDAD DE ESTILOS
- BUENAS PRÁCTICAS Y RESPONSIVE
- BIBLIOGRAFÍA

CSS: Posicionamiento y Flujo (static)

- En CSS, todos los elementos HTML siguen un flujo normal de colocación.
- Es el comportamiento por defecto:

```
div {  
  display: block; /* por defecto */  
}  
span {  
  display: inline; /* por defecto */  
}
```

- El navegador ubica cada elemento según su tipo:
 - **Elementos de bloque (block)**
 - Ocupan todo el ancho disponible.
 - Empiezan en una nueva línea.
 - Ejemplos: <div>, <p>, <h1>...<h6>.
 - **Elementos en línea (inline)**
 - Ocupan solo el espacio de su contenido.
 - Se colocan uno al lado del otro. Ejemplos: , <a>, .

CSS: Modos de posicionamiento

- La propiedad `position` define cómo se ubica un elemento en la página.

- **static (por defecto)**

- Sigue el flujo normal del documento.
- No acepta `top`, `left`, `right`, `bottom`.

- **relative**

- Mantiene su espacio en el flujo.
- Se puede mover respecto a su posición original.

```
.caja1 { position: relative; top: 10px; left: 20px; }
```

- **absolute**

- Se posiciona respecto al contenedor más cercano con `position: relative | absolute | fixed | sticky`.
- Sale del flujo normal.

```
.caja2 { position: absolute; bottom: 0; right: 0; }
```

- **fixed**

- Fijo en la ventana del navegador.
- No se mueve aunque haya scroll.

```
.menu { position: fixed; top: 0; left: 0; }
```

- **Sticky (más indicado en menús o encabezados, mejora la navegación)**

- Mezcla de `relative` y `fixed`.
- Se comporta como `relative` hasta que se alcanza un umbral, entonces queda fijo.

```
header { position: sticky; top: 0; }
```

CSS: Propiedades de posicionamiento y capas

- Coordenadas

- top, right, bottom, left
- Funcionan solo si el elemento no es static.

```
.caja { position: absolute; top: 20px; left: 30px; }
```

- Capas (z-index)

- Controla qué elemento se muestra por encima de otro.
- Valor numérico (positivo o negativo).
- Requiere que el elemento tenga position <> static.

```
.caja1 { position: relative; z-index: 10; }  
.caja2 { position: relative; z-index: 5; }
```

CSS: Posicionamiento Accesible. Buenas prácticas

- Orden lógico en el DOM
 - Los lectores de pantalla leen en el orden del HTML, no en el orden visual.
 - No usar absolute/z-index para reorganizar elementos importantes ya que puede romper la narrativa.
- Elementos fijos (fixed, sticky)
 - Útiles para menús y cabeceras.
 - Evitar que tapen contenido o botones importantes.
 - Revisar en móvil: un fixed mal usado puede cubrir medio viewport.
- Claridad visual
 - Asegurar que elementos reposicionados siguen teniendo contraste suficiente y no se solapan con otros.
 - Mantener jerarquía semántica: si es navegación, debe seguir siendo <nav>; si es título, debe seguir siendo <h1>.

```
header {  
  position: sticky;  
  top: 0;  
  background: white;  
  z-index: 100; /* visible pero no oculta contenido */  
}
```

CSS: Posicionamiento Accesible. Buenas prácticas

Qué ocurre: al hacer scroll

```
body {  
  margin: 0;  
  font-family: Arial, sans-serif;  
}  
header {  
  position: sticky;  
  top: 0;  
  background: #333;  
  color: white;  
  padding: 20px;  
  z-index: 100;  
}  
main {  
  padding: 20px;  
  margin-top: 200px; /* espacio para que el texto no quede debajo */  
}
```

Mi sitio web

```
header {  
  position: fixed;
```

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Proin in...

Sigue bajando con scroll para ver el efecto.

Texto de relleno para generar scroll...

Mi sitio web

```
header {  
  position: sticky;
```

Contenido principal

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Proin in...

Sigue bajando con scroll para ver el efecto.

“Si usas position: fixed en el header y no reservas espacio arriba, tu contenido desaparecerá debajo del header.”

CSS: Ejercicio

Crea una página sencilla con los siguientes elementos y aplica lo aprendido sobre flujo y posicionamiento:

- Cabecera fija
 - Un `<header>` con un título centrado.
 - Usa `position: fixed; top:0;` → debe quedar siempre visible.
 - Reserva espacio en el contenido para que no quede oculto.
- Banner con texto centrado
 - Un `<section>` a modo de banner.
 - El texto debe aparecer centrado vertical y horizontalmente dentro del banner (usa `position: relative` en el contenedor y `absolute` en el texto).
- Caja destacada superpuesta
 - Dentro del `<main>`, coloca una `<div class="destacada">`.
 - Esta caja debe “flotar” sobre el resto con `position: absolute;` y un `z-index` mayor.
- Botón pegajoso
 - Añade un botón con `position: sticky; bottom: 20px;`.
 - Al hacer scroll debe mantenerse visible en la parte baja del contenedor.
- El orden del HTML debe ser lógico para la lectura, aunque visualmente cambies posiciones.
- El header fijo no debe tapar el contenido inicial.
- Asegura contraste entre texto y fondo en el banner y la caja destacada.

CSS: Ejercicio: HTML base

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <title>Ejercicio Posicionamiento y Flujo</title>
  <link rel="stylesheet" href="estilos.css">
</head>
<body>
  <header>
    <h1>Mi sitio web</h1>
  </header>

  <section class="banner">
    <h2>Bienvenido a la página</h2>
  </section>

  <main>
    <p>Lorem ipsum dolor sit amet, consectetur adipiscing elit. Vivamus ac magna eget urna cursus placerat. Donec euismod, nunc vel tincidunt tempus, augue risus.</p>
    <p>Sigue bajando para ver el botón sticky y la caja destacada.</p>

    <div class="destacada">
      <h3>Caja destacada</h3>
      <p>Esta caja debe superponerse al contenido.</p>
    </div>

    <p style="height: 800px;">Texto de relleno para hacer scroll...</p>

    <button class="boton-sticky">Acción rápida</button>
  </main>
</body>
</html>
```

CSS: Ejercicio: Solución stylesheet

```
body {
  margin: 0;
  font-family: Arial, sans-serif;
}
/* Cabecera fija */
header {
  position: fixed;
  top: 0;
  left: 0;
  width: 100%;
  background: #333;
  color: white;
  padding: 20px;
  text-align: center;
  z-index: 1000;
}
/* Reservar espacio para que el header no tape contenido */
main {
  margin-top: 80px; /* altura aproximada del header */
  padding: 20px;
}
/* Banner con texto centrado */
.banner {
  position: relative;
  background: lightblue;
  height: 200px;
}
```

```
.banner h2 {
  position: absolute;
  top: 50%;
  left: 50%;
  transform: translate(-50%, -50%);
  margin: 0;
}
/* Caja destacada superpuesta */
.destacada {
  position: absolute;
  top: 300px;
  right: 20px;
  width: 250px;
  background: orange;
  color: black;
  padding: 15px;
  border-radius: 8px;
  z-index: 500;
  box-shadow: 0 4px 8px rgba(0,0,0,0.3);
}
/* Botón sticky */
.boton-sticky {
  position: sticky;
  bottom: 20px;
  padding: 10px 20px;
  background: crimson;
  color: white;
  border: none;
  border-radius: 6px;
  cursor: pointer;
}
```

CSS: Ejercicio. Solución visual

Mi sitio web

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Vivamus ac magna eget urna curabitur tincidunt tempus, augue risus.

Sigue bajando para ver el botón sticky y la caja destacada.

Texto de relleno para hacer scroll...

Caja destacada

Esta caja debe superponerse al contenido.

Acción rápida

CONTENIDO

- INTRODUCCIÓN
- SELECTORES Y SINTAXIS
- PROPIEDADES FUNDAMENTALES
- MODELO DE CAJA (BOX MODEL)
- POSICIONAMIENTO Y FLUJO
- **LAYOUTS: FLEXBOX Y GRID**
- PRIORIDAD DE ESTILOS
- BUENAS PRÁCTICAS Y RESPONSIVE
- BIBLIOGRAFÍA

CSS: Layouts. Introducción

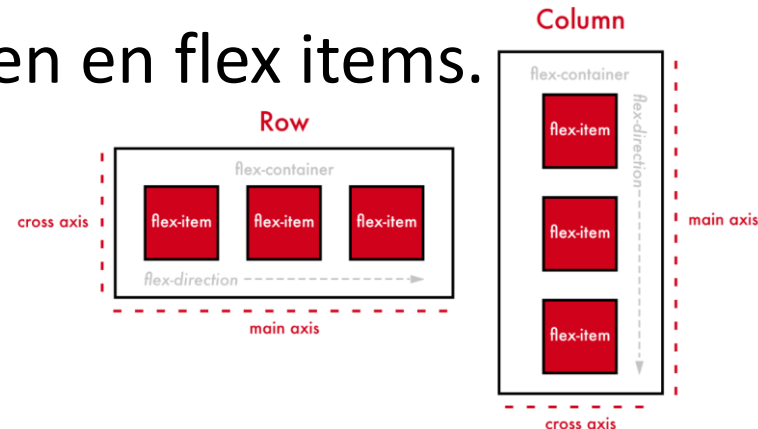
- Breve historia:
 - Layout con tablas en HTML
 - Float y clear en CSS
 - inline-block y position
- Limitaciones
 - Dificultad de centrar verticalmente
 - Problemas de espacio y alineación
 - Poco adaptables a diseños responsive
- **Flexbox**: pensado para organizar elementos en una dimensión (fila o columna).
- **Grid**: diseñado para organizar elementos en dos dimensiones (filas y columnas). Son más simples, potentes y responsivos.
- Permiten mantener el orden semántico:
header-nav-main-footer



CSS: Layouts. Fundamentos del Flexbox

- Se aplica en el contenedor con **display: flex;**
- Los elementos hijos se convierten en flex items.

```
.container {  
  display: flex;  
  flex-direction: row;    /* horizontal (por defecto) */  
  flex-direction: column; /* vertical */  
}
```



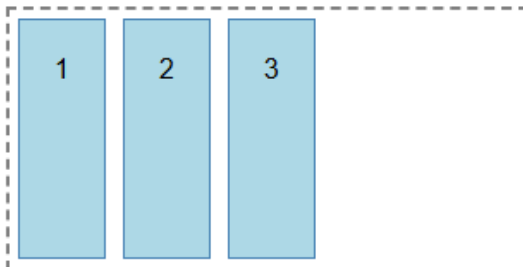
- Asegurar suficiente espacio (gap) entre los elementos del container

```
.container {  
  justify-content: center; /* alinea en el eje principal */  
  align-items: flex-start; /* alinea en el eje secundario */  
  gap: 20px;             /* espacio entre items */  
}
```

CSS: Layouts. Ejemplo Flexbox

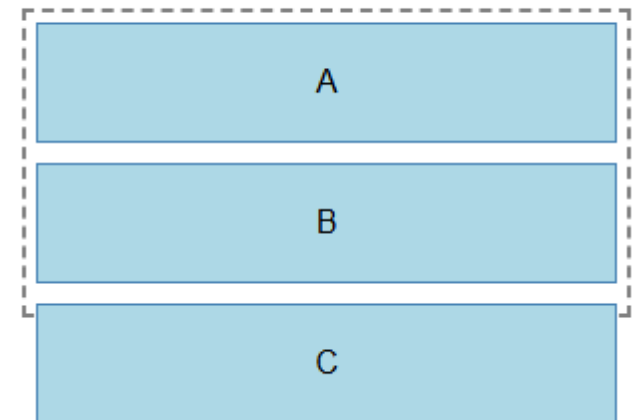
```
.container {  
  display: flex;  
  border: 2px dashed gray;  
  margin-bottom: 30px;  
  height: 150px;  
  width: 300px;  
}  
.row {  
  flex-direction: row; /* eje principal horizontal */  
}  
.column {  
  flex-direction: column; /* eje principal vertical */  
}  
.item {  
  background: lightblue;  
  border: 1px solid steelblue;  
  padding: 20px;  
  margin: 5px;  
  text-align: center;  
  flex: 0 0 auto; /* no crecer, no encoger */  
}
```

flex-direction: row



```
<h2>flex-direction: row</h2>  
<div class="container row">  
  <div class="item">1</div>  
  <div class="item">2</div>  
  <div class="item">3</div>  
</div>  
  
<h2>flex-direction: column</h2>  
<div class="container column">  
  <div class="item">A</div>  
  <div class="item">B</div>  
  <div class="item">C</div>  
</div>
```

flex-direction: column



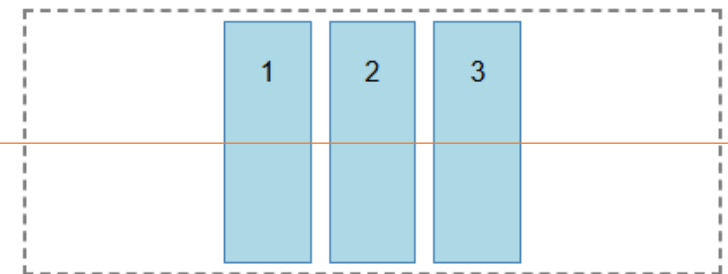
CSS: Layouts. Ejemplo Flexbox

```
.container {
  display: flex;
  border: 2px dashed gray;
  margin-bottom: 30px;
  height: 150px;
  width: 400px;
  background: #f9f9f9;
}
/* --- Justify-content --- */
.justify {
  justify-content: center; /* mueve en horizontal (eje
principal) */
}
/* --- Align-items --- */
.align {
  align-items: center; /* mueve en vertical (eje secundario) */
}
.item {
  background: lightblue;
  border: 1px solid steelblue;
  padding: 20px;
  margin: 5px;
  text-align: center;
}
.alto {
  padding: 50px 20px; /* uno más alto para ver mejor la
alineación */
}
```

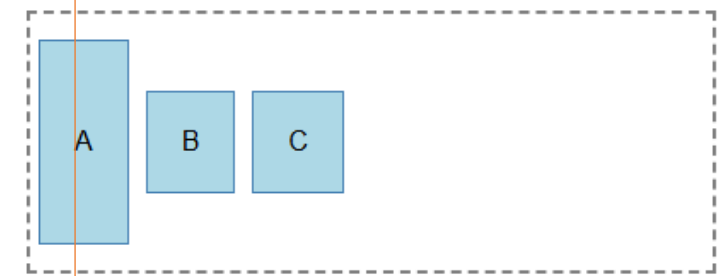
```
<h2>justify-content (eje principal)</h2>
<div class="container row justify">
  <div class="item">1</div>
  <div class="item">2</div>
  <div class="item">3</div>
</div>
```

```
<h2>align-items (eje secundario)</h2>
<div class="container row align">
  <div class="item alto">A</div>
  <div class="item">B</div>
  <div class="item">C</div>
</div>
```

justify-content (eje principal)



align-items (eje secundario)



CSS: Layout. Flexbox avanzado

- flex-wrap: Controla si los items se mantienen en una sola línea o pueden saltar a varias.

```
display: flex;
flex-wrap: wrap; /* los items bajan si no caben */
```

- Relativo a los hijos (flex items)
 - flex-grow → cuánto puede crecer un item para ocupar espacio libre.

```
flex-basis: 100px; /* base de 100px */
```

- flex-shrink → cuánto puede encoger un item si falta espacio.

```
flex-shrink: 0; /* este no se encoge */
```

- flex-basis → tamaño base del item antes de crecer o encoger.

```
flex: 2 0 100px; /* ocupa el doble que grow1 */
```

- Atajo flex → combina grow shrink basis.

```
/* --- Atajo flex (grow shrink basis) --- */
flex: 1 1 100px; /* crece normal */
```

- align-self: Permite a un item sobrescribir el align-items del contenedor.

```
align-items: flex-start;
...
align-self: flex-end;
```

CSS: Layout. Flexbox avanzado. Ejemplo

```
.container {  
  display: flex;  
  border: 2px dashed gray;  
  margin-bottom: 30px;  
  padding: 10px;  
  height: 120px;  
  background: #f9f9f9;  
}  
  
.item {  
  background: lightblue;  
  border: 1px solid steelblue;  
  padding: 10px;  
  margin: 5px;  
  text-align: center;  
}  
  
/* --- flex-wrap --- */  
.wrap {  
  /* los items saltan de línea */  
  flex-wrap: wrap;  
}  
  
.wrap .item {  
  /* cada item mide 100px de base */  
  flex: 0 0 100px;}
```

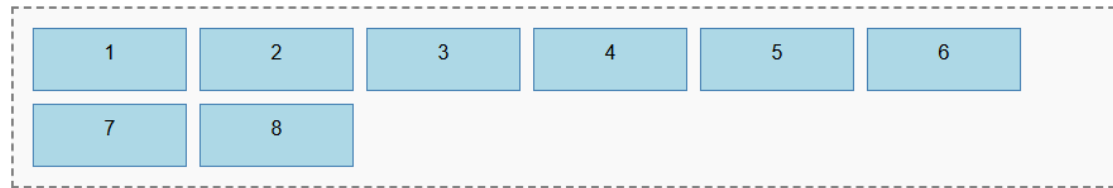
```
/* --- flex-grow, flex-shrink, flex-basis --- */  
.basis .grow {  
  flex-grow: 2; /* crece más */  
  flex-basis: 100px; /* base de 100px */  
  background: lightgreen;  
}  
  
.basis .shrink {  
  flex-shrink: 0; /* no se encoge */  
  flex-basis: 200px;  
  background: lightcoral;  
}  
  
.basis .fixed {  
  flex-basis: 150px; /* tamaño fijo de base */  
  background: lightyellow;  
}  
  
/* --- Atajo flex (grow shrink basis) --- */  
.atajo .flex1 {  
  flex: 1 1 100px; /* crece normal */  
}  
  
.atajo .flex2 {  
  flex: 2 1 100px; /* ocupa el doble que flex1 */  
  background: lightpink;  
}  
  
/* --- align-self --- */  
.align {  
  align-items: flex-start;  
}  
  
.especial {  
  align-self: flex-end;  
  background: lightsalmon;  
}
```

CSS: Layout. Flexbox avanzado. Ejemplo

<h2>Flex-wrap</h2>

```
<div class="container wrap">
  <div class="item">1</div>
  <div class="item">2</div>
  <div class="item">3</div>
  <div class="item">4</div>
  <div class="item">5</div>
  <div class="item">6</div>
  <div class="item">7</div>
  <div class="item">8</div>
</div>
```

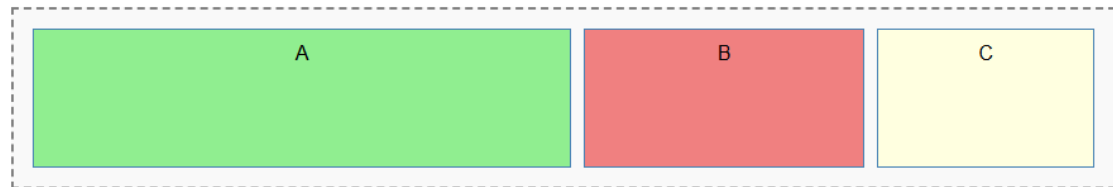
Flex-wrap



<h2>Flex-grow, Flex-shrink y Flex-basis</h2>

```
<div class="container basis">
  <div class="item grow">A</div>
  <div class="item shrink">B</div>
  <div class="item fixed">C</div>
</div>
```

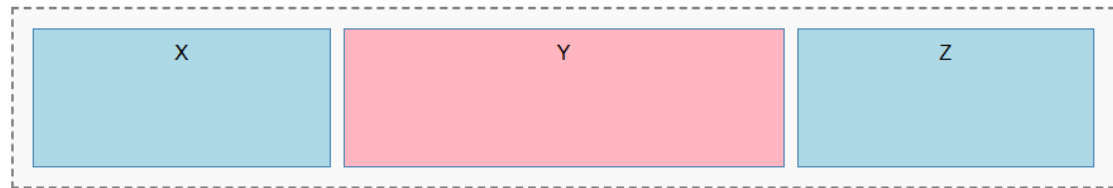
Flex-grow, Flex-shrink y Flex-basis



<h2>Atajo con Flex</h2>

```
<div class="container atajo">
  <div class="item flex1">X</div>
  <div class="item flex2">Y</div>
  <div class="item flex1">Z</div>
</div>
```

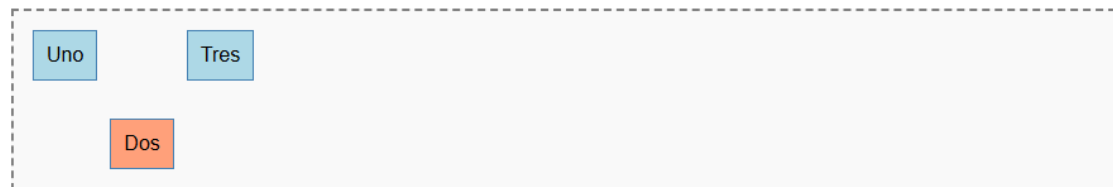
Atajo con Flex



<h2>Align-self</h2>

```
<div class="container align">
  <div class="item">Uno</div>
  <div class="item especial">Dos</div>
  <div class="item">Tres</div>
</div>
```

Align-self



CSS: Layouts. Fundamentos del Grid

- Sistema de maquetación bidimensional: controla filas y columnas a la vez.
- Ideal para layouts completos de página o estructuras de rejilla.
- Complementa a Flexbox (una dimensión, columnas o filas).
- Para activar el Grid, hay que definirlo en el container:
- A continuación se definen las filas y las columnas

```
.container {  
  display: grid;  
  grid-template-columns: 100px 200px auto;  
  grid-template-rows: 150px auto;  
  gap: 10px;  
}
```

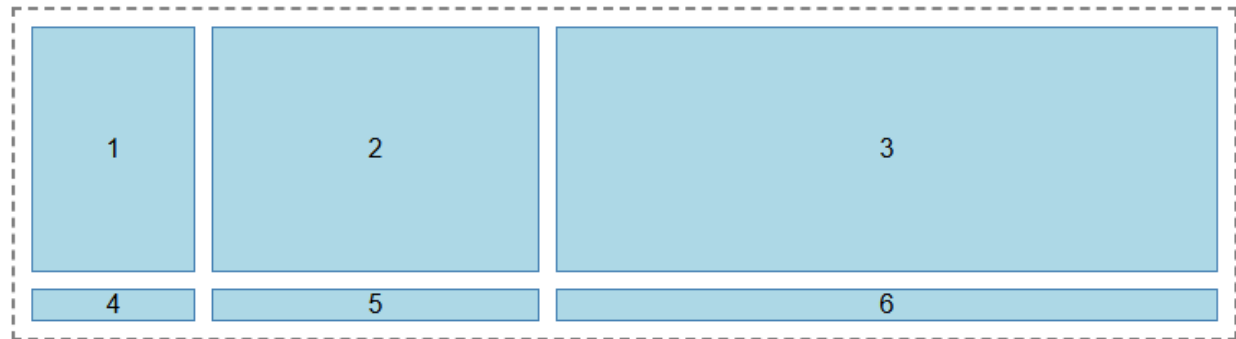
CSS: Layouts. Fundamentos del Grid

- Grid trabaja con las siguientes unidades:
 - px: tamaño fijo
 - %: relativo al contenedor
 - fr: “fracciones”, reparte espacio proporcionalmente

```
.grid {  
  display: grid;  
  grid-template-columns: 100px 200px auto; /* tres columnas */  
  grid-template-rows: 150px auto;          /* dos filas */  
  gap: 10px;                               /* espacio entre celdas */  
  border: 2px dashed gray;  
  padding: 10px;  
}
```

```
.item {  
  background: lightblue;  
  border: 1px solid steelblue;  
  display: flex;  
  align-items: center;  
  justify-content: center;  
}
```

```
<div class="grid">  
  <div class="item">1</div>  
  <div class="item">2</div>  
  <div class="item">3</div>  
  <div class="item">4</div>  
  <div class="item">5</div>  
  <div class="item">6</div>  
</div>
```



Si quieres que fr o % funcione en filas, se debe poner una altura al contenedor.

CSS: Layouts. Grid avanzado

- Grid Areas

- Permite nombrar zonas de la cuadrícula y asignarles elementos.

```
.container {  
  display: grid;  
  grid-template-areas:  
    "header header"  
    "menu   main"  
    "footer footer";  
  grid-template-columns: 150px 1fr;  
  grid-template-rows: 80px 1fr 60px;  
}
```

```
.header { grid-area: header; }  
.menu   { grid-area: menu;   }  
.main   { grid-area: main;   }  
.footer { grid-area: footer; }
```

- Posicionamiento manual

- Con grid-row y grid-column se puede colocar un item en coordenadas específicas:

```
.item1 {  
  grid-column: 1 / 3; /* comienza en la columna 1 y termina en la 3. Ocupa 2 columnas */  
  grid-row: 1 / 2;    /* comienza en la fila 1 y termina en la 2. Ocupa la primera fila */  
}
```

- Funciones útiles:

- repeat(n, tamaño): repite columnas o filas.
- minmax(min, max): define tamaños flexibles con límites.
- auto-fit / auto-fill → crean layouts responsivos automáticos.

```
grid-template-columns: repeat(3, 1fr); /* 3 columnas iguales */  
grid-template-columns: repeat(3, minmax(150px, 1fr));
```

- Mantener el orden semántico en HTML: Grid cambia la posición visual, no la del DOM.
- Evitar diseños donde la lectura con tabulador no siga un orden lógico.

CSS: Layouts. Grid avanzado. Ejemplo

```
body {  
  margin: 0;  
  font-family: Arial, sans-serif;  
}  
  
.grid {  
  display: grid;  
  grid-template-areas:  
    "header header"  
    "menu   main"  
    "aside  main"  
    "footer footer";  
  grid-template-columns: 200px 1fr;  
  grid-template-rows: 80px repeat(2, minmax(150px, auto)) 60px;  
  gap: 10px;  
  height: 100vh; /* toda la altura de la ventana */  
}  
  
.header { grid-area: header; background: lightblue; }  
.menu   { grid-area: menu;   background: lightgreen; }  
.main   { grid-area: main;   background: lightyellow; }  
.aside  { grid-area: aside;  background: lightcoral; }  
.footer { grid-area: footer; background: lightgray; }  
  
/* Ejemplo de posicionamiento manual extra */  
.main {  
  grid-column: 2 / 3; /* explícitamente en la segunda columna */  
  grid-row: 2 / 4;    /* ocupa desde la fila 2 hasta la 3 */  
}
```

CSS: Layouts. Grid avanzado. Ejemplo

Header

Menú

Contenido principal

Aside

Footer

```
<div class="grid">  
  <header class="header">Header</header>  
  <nav class="menu">Menú</nav>  
  <main class="main">Contenido principal</main>  
  <aside class="aside">Aside</aside>  
  <footer class="footer">Footer</footer>  
</div>
```

CONTENIDO

- INTRODUCCIÓN
- SELECTORES Y SINTAXIS
- PROPIEDADES FUNDAMENTALES
- MODELO DE CAJA (BOX MODEL)
- POSICIONAMIENTO Y FLUJO
- LAYOUTS: FLEXBOX Y GRID
- **PRIORIDAD DE ESTILOS**
- BUENAS PRÁCTICAS Y RESPONSIVE
- BIBLIOGRAFÍA

CSS: Prioridad de estilos. Introducción

- Orden natural en CSS
 - Estilos del navegador: lo que pone por defecto (ej. enlaces azules).
 - Estilos del autor: los que escribimos en nuestra hoja CSS.
 - Estilos en línea (`style=""` en HTML) → tienen más fuerza.
- Cuando hay conflicto, gana el selector más específico. El orden sería este:
 - Etiqueta (p) < Clase (.btn) < ID (#menu) < Estilos en línea

```
p { color: black; }      /* básico */
.texto { color: blue; }  /* gana sobre el anterior */
#intro { color: red; }   /* gana sobre todos */
```

- Si dos reglas tienen la **misma especificidad**, gana la que está **más abajo** en la hoja de estilos.
- Usar clases en lugar de IDs cuando sea posible → facilita reutilización.

CSS: Prioridad de estilos. Ejercicio

- Dado el siguiente código HTML

```
<main>
<h1 id="titulo" class="destacado">Mi página</h1>
<p class="texto">Bienvenido a mi sitio web.</p>
<p id="intro" class="texto">Este es el párrafo introductorio.</p>
<section>
  <p>Otro párrafo dentro de una sección.</p>
</section>
</main>
```

- Aplica estilos en conflicto:
 - Un estilo general para p (ej. color gris).
 - Un estilo para .texto (ej. azul).
 - Un estilo para #intro (ej. rojo).
 - Un estilo para main p (ej. verde).
- Comprueba qué regla gana en cada párrafo.
- Cambia el orden de las reglas en el CSS y observa qué ocurre si tienen la misma especificidad.
- Añade al h1:
 - Un estilo de etiqueta h1 {}
 - Un estilo de clase .destacado {}
 - Un estilo de ID #titulo {}
 - ¿Qué estilo se aplica al título?.
- Probar color: inherit; en un párrafo para heredar el color del main.

CONTENIDO

- INTRODUCCIÓN
- SELECTORES Y SINTAXIS
- PROPIEDADES FUNDAMENTALES
- MODELO DE CAJA (BOX MODEL)
- POSICIONAMIENTO Y FLUJO
- LAYOUTS: FLEXBOX Y GRID
- PRIORIDAD DE ESTILOS
- **BUENAS PRÁCTICAS Y RESPONSIVE**
- BIBLIOGRAFÍA

CSS: Buenas prácticas

- Nombres claros y semánticos

```
/* ✗ Evitar */  
.rojo { color: red; }  
  
/* ✓ Mejor */  
.btn-primario { color: white; background: blue; }
```

- Reutilización de estilos

```
/* ✗ Estilos repetidos */  
h1 { font-family: Arial; }  
p { font-family: Arial; }  
  
/* ✓ Centralizar */  
body { font-family: Arial, sans-serif; }
```

- Organización del código

```
/* --- Botones --- */  
.btn { padding: 10px; border-radius: 5px; }  
.btn-primario { background: blue; color: white; }  
  
/* --- Menú --- */  
.menu { display: flex; gap: 10px; }
```

CSS: Buenas prácticas.

Un buen CSS no solo funciona, también es limpio, mantenible y se adapta a cualquier pantalla.

```
/* Repetición innecesaria */
h1 { font-family: Arial; font-size: 24px; color: red; }
p { font-family: Arial; font-size: 16px; color: red; }
div { font-family: Arial; font-size: 14px; color: red; }

/* Nombres poco claros */
.rojo { color: red; }
.grande { font-size: 30px; }
```



```
/* Centralizar estilos comunes */
body { font-family: Arial, sans-serif; color: red; }

/* Jerarquía semántica */
h1 { font-size: 2rem; }
p { font-size: 1rem; }

/* Clases claras y reutilizables */
.btn-primario { background: blue; color: white; }
```



Consecuencias:

- Código largo y difícil de mantener.
- Cambiar un estilo obliga a modificar varias líneas.
- Poco reutilizable, nada semántico.

Ventajas:

- Código más corto y mantenible.
- Fácil de escalar: cambios globales en un solo sitio.
- Mejor accesibilidad y consistencia.

CSS: Diseño Responsive. Unidades relativas

- Utilización de unidades relativas:
 - Permiten que el diseño se adapte al dispositivo.
 - Mejor que los valores fijos en px, que no responden a cambios de pantalla ni de zoom.

```
.container {  
  width: 80%;           /* relativo al padre */  
  font-size: 1.2em;     /* relativo al tamaño de fuente del padre */  
  padding: 2rem;        /* relativo al tamaño raíz (html) */  
  height: 50vh;         /* 50% de la altura de la ventana */  
}
```

- % : relativo al tamaño del contenedor padre.
- em : relativo al tamaño de fuente del elemento padre.
- rem : relativo al tamaño de fuente raíz (html).
- vh / vw : relativo al alto / ancho del viewport.

CSS: Diseño Responsive. Media Queries

- Permiten aplicar estilos condicionales según el ancho de pantalla u otras características del dispositivo.

```
@media (max-width: 600px) {  
    /* reglas para pantallas pequeñas */  
}
```

- max-width: 600px : se aplica cuando el ancho de la ventana es 600px o menos.
- min-width: 1024px : se aplica en pantallas desde 1024px en adelante.

```
.menu {  
    display: flex;  
    flex-direction: row; /* horizontal en escritorio */  
}  
@media (max-width: 600px) {  
    .menu {  
        flex-direction: column; /* vertical en móvil */  
    }  
}
```

CSS: Buenas prácticas en Responsive.

- Mobile First: Se comienza con estilos para pantallas pequeñas y se amplía con media queries.
 - La web será usable en cualquier dispositivo y no dependerá de recortar un diseño de escritorio.

```
/* Base: móvil */
.menu {
  display: flex;
  flex-direction: column;
}
/* Escritorio en adelante */
@media (min-width: 768px) {
  .menu { flex-direction: row; }
}
```

- Layout fluidos: Mejor utilizar unidades relativas que valores fijos en px.

```
.container {
  width: 90%; /* se adapta al ancho disponible */
  max-width: 1200px; /* límite en pantallas grandes */
}
```

- Imágenes flexibles: las imágenes se ajustarán al contenedor sin deformarse

```
img {
  max-width: 100%;
  height: auto;
}
```

CONTENIDO

- INTRODUCCIÓN
 - SELECTORES Y SINTAXIS
 - PROPIEDADES FUNDAMENTALES
 - MODELO DE CAJA (BOX MODEL)
 - POSICIONAMIENTO Y FLUJO
 - LAYOUTS: FLEXBOX Y GRID
 - PRIORIDAD DE ESTILOS
 - BUENAS PRÁCTICAS Y RESPONSIVE
 - **BIBLIOGRAFÍA**
- 

BIBLIOGRAFÍA

- E. Robson, E. Freeman
Head First HTML and CSS
O'Reilly (2012)
- HTML Reference
<http://www.w3schools.com/tags/>
- CSS Reference
<http://www.w3schools.com/cssref/>

